CS3160 Lab 4

Instruction Decode and Disassembly

What you will do today

Last week your simulator could load a program and show you the bytes. Today we will start making our simulator "understand" those bytes.

In this lab, you will write the `decode()` function, which turns a 32-bit word into a struct saying what instruction it is, what its operands are. You will also write the `disasm()` function, which turns that struct into a human-readable text. By the end of the session your simulator can print any program as assembly — and you will be able to check it against `objdump`.

Next week, we will start building the processor (simulator), and a disassembler you trust will help debug it. So it is important to get this lab right.

1 A RISC-V instruction

Every instruction is exactly 32 bits, and the low 7 bits are always the opcode. This makes RISC-V easy to decode. The opcode tells you the *format*, and the format tells you which other fields exist:

Format	Has	Immediate	Example
R	rd, rs1, rs2	none	`add rd, rs1, rs2`
I	rd, rs1	12-bit, signed	`addi`, `lw`, `jalr`
S	rs1, rs2	12-bit, split in two	`sw`
B	rs1, rs2	13-bit, scrambled	`beq`
U	rd	20-bit, high bits	`lui`, `auipc`
J	rd	21-bit, scrambled	`jal`

`funct3` (bits 14–12) and `funct7` (bits 31–25) then separate the instructions that share an opcode — `add` and `sub` differ only in `funct7`.

The immediates are already written

The immediate is the one field which the RISC-V isa scrambles: a branch offset arrives in four pieces and out of order. This is a trade-off to simplify the hardware, but makes the decoder slightly more complex.

So those five functions have been given (see top of `src/decode.cpp`):

```
imm_i    imm_s    imm_b    imm_u    imm_j
```

Each takes the raw word and returns the immediate **already sign-extended into a** `u32`, ready to use. Read them carefully.

▶ Sign-extended *means a negative immediate arrives as a* `u32` *whose top bits are all ones. That is exactly what you want:* `pc + d.imm` *then lands in the right place for a backward branch without any special case, because unsigned arithmetic wraps.*

2 Part 1: `decode()`

Fill in `d.op`, `d.fmt`, `d.rd`, `d.rs1`, `d.rs2` and `d.imm`. Leave `d.pc` as the pc you were handed.

An encoding you do not recognize is `OP_INVALID` with `FMT_NONE`, not a crash. When some other module (which you will develop in future) tries to execute such an instruction, it will be able to print a helpful error message.

Work opcode by opcode and run `make test` after each. The unit tests are named for the instruction they decode, so the list of failures is your list of what to do next. A sensible order:

1. `addi` and the rest of the I-type arithmetic — the simplest format, and it gets the register fields right.
2. R-type: `add`, `sub`, and the shifts.
3. Loads and stores — same immediates as above, different formats.
4. Branches and jumps — the scrambled immediates, all handed to you.
5. `lui`, `auipc`, and the M extension (`mul`, `div`).

3 Part 2: `disasm()`

Now render a `Decoded` as text. The format is the one `objdump` prints with `-M no-aliases`, because that is what we compare against:

```
addi t0, zero, 1
lw t0, 8(t1)
beq t0, t1, 0x80000010
```

Three things to get right:

- **Register names are ABI names.** `t0`, not `x5`. `reg_name()` at the top of the file does this.
- **Loads and stores use** `offset(base)`, not three commas.
- **Branches and jumps print their target address**, not their offset. You can work it out because `d.pc` is right there in the struct: the target is `d.pc + d.imm`.

`format()` above is a `printf` that returns a `std::string`, which is usually the shortest way to write each case.

Why `no-aliases`

Real `objdump` prints `nop` where the encoding says `addi zero, zero, 0`, and `ret` where it says `jalr zero, 0(ra)`. Those are *aliases* — friendlier names for encodings that do something recognizable. We turn them off on both sides for ease of implementation. Print what the encoding actually says, every time.

4 Checking your work

```
make          # build build/sim
make test     # build, then unit, then programs
make unit     # just the unit tests
```

The program tier disassembles the first 512 bytes of each of five programs and compares against ours, line for line. It will tell you the first line that differs and what we expected there.

You can also look at any program yourself:

```
build/sim --norun --disasm=0x80000000:0x80000100 tests/images/1-even.
    r5ob
```

Checking against `objdump`

Once the tests pass, try the following:

```
cd programs && make
riscv-none-elf-objdump -d -M no-aliases bins/asms/1-even.r5o
```

and compare it against your own output for the same range. `objdump` is the golden reference: it is part of the toolchain that produced the file. If your code's output and it disagree, your code is wrong, and the instruction that disagrees is the one to fix.

▶ *Do not skip this because the tests passed. The tests cover 512 bytes of five programs, which is a lot but not everything; `objdump` covers whatever you point it at. An encoding you got wrong, but did not appear in those five programs may come back to haunt you in a future lab.*

5 What to hand in

Your folder is collected at the end of the session and **only what is in it at that moment is graded**:

1. `details.txt` — name, roll number, machine number.
2. `src/decode.cpp` and `src/disasm.cpp`.